

Curso desarrollo de aplicaciones con tecnologías web

UF1841. Elaboración de documentos web mediante lenguajes de marcas

Lenguajes de marcado generales

Origen de los lenguajes de marcado generales: SGML y XML

Características generales de los lenguajes de marcado

Estructura general de un documento con lenguaje de marcado

Documentos válidos y bien formados. Esquemas

Origen de los lenguajes de marcado generales: SGML y XML

Los lenguajes de marcado tienen su origen en la necesidad de estructurar información de manera que pueda ser interpretada tanto por humanos como por máquinas.

- **SGML (Standard Generalized Markup Language):** Desarrollado en los años 80, es un estándar ISO que define reglas para crear lenguajes de marcado personalizados. Es muy flexible y potente, pero también complejo, lo que limita su adopción para aplicaciones más prácticas.
- **XML (Extensible Markup Language):** Derivado de SGML, XML se diseñó en los años 90 para ser más sencillo y ampliamente utilizable. Su objetivo es estructurar datos de forma que puedan ser compartidos y procesados de manera consistente en diferentes sistemas. XML conserva la esencia de SGML, pero con menos complejidad.

Ambos lenguajes son la base de muchas tecnologías actuales, como HTML, XHTML y otros lenguajes específicos.

Características generales de los lenguajes de marcado

Los lenguajes de marcado comparten las siguientes características principales:

- **Estructuración de datos:** Permiten organizar información de forma jerárquica y lógica.
- **Uso de etiquetas:** Utilizan etiquetas (o “marcas”) para definir elementos y su contenido.
- **Portabilidad:** Pueden ser interpretados en múltiples plataformas y sistemas.
- **Legibilidad:** Son entendibles tanto para humanos como para máquinas.
- **Extensibilidad:** Especialmente en XML, permiten definir etiquetas y estructuras personalizadas.
- **Separación de contenido y presentación:** Facilitan mantener el contenido independiente de su formato visual.

Estructura general de un documento con lenguaje de marcado

Un documento en un lenguaje de marcado generalmente incluye los siguientes componentes:

1. Metadatos e instrucciones de proceso: Los metadatos proporcionan información sobre el documento (autor, fecha, descripción, etc.), mientras que las instrucciones de proceso guían a la aplicación sobre cómo manejar el contenido.

Ejemplo en XML:

```
<?xml version="1.0" encoding="UTF-8"?>
```

2. Codificación de caracteres. Caracteres especiales (escape): La codificación de caracteres garantiza que el texto se represente correctamente en cualquier sistema. En XML, el estándar más usado es UTF-8. Los caracteres especiales se escapan para evitar conflictos.

Ejemplo de escape:

```
&lt; (menor que) -> <  
&gt; (mayor que) -> >  
&amp; (ampersand) -> &
```

3. Etiquetas o marcas: Las etiquetas son la base de un lenguaje de marcado. Se utilizan para delimitar el inicio y el final de los elementos.

```
<nombre>Juan</nombre>
```

4. Elementos: Un elemento incluye una etiqueta de inicio, contenido y una etiqueta de cierre. Los elementos pueden estar anidados.

```
<persona>  
  <nombre>Juan</nombre>  
  <edad>30</edad>  
</persona>
```

5. Atributos: Los atributos proporcionan información adicional sobre los elementos y se escriben dentro de la etiqueta de inicio.

```
<persona id="123" genero="masculino">  
  <nombre>Juan</nombre>  
</persona>
```

6. Comentarios: Los comentarios se usan para agregar notas que no serán procesadas.

```
<!-- Este es un comentario -->
```

Documentos válidos y bien formados. Esquemas

1. Documentos bien formados: Cumplen con las reglas sintácticas básicas del lenguaje, como el uso correcto de etiquetas, atributos y anidaciones.

Ejemplo bien formado:

```
<libro>
  <titulo>XML en acción</titulo>
</libro>
```

Ejemplo mal formado (sin cierre de etiqueta):

```
<libro>
  <titulo>XML en acción</titulo>
```

2. Documentos válidos: Además de estar bien formados, deben cumplir con un esquema que define las estructuras y elementos permitidos. Los esquemas más comunes son **DTD (Document Type Definition)** y **XML Schema**.

Ejemplo de un esquema simple en DTD:

```
<!DOCTYPE libro [
  <!ELEMENT libro (titulo, autor)>
  <!ELEMENT titulo (#PCDATA)>
  <!ELEMENT autor (#PCDATA)>
]>
```

¿Qué es XML Schema?

Un XML Schema (XSD) es un archivo que define las reglas para un documento XML. Sirve para comprobar que el XML es correcto (válido). Básicamente, es como una “guía” que explica:

- Qué elementos pueden aparecer.
- En qué orden deben aparecer.
- Qué atributos pueden tener.
- Qué tipo de datos (texto, números, fechas, etc.) deben contener.

Ejemplo Simple: XML y su Schema

1. Archivo XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<persona xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="persona.xsd">
  <nombre>Juan</nombre>
  <edad>30</edad>
</persona>
```

En este XML:

- Tenemos un elemento raíz <persona>.
- Contiene dos elementos hijos: <nombre> y <edad>.
- El atributo xsi:noNamespaceSchemaLocation dice que las reglas están en un archivo llamado persona.xsd.

2. Archivo XML Schema (persona.xsd):

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- Definimos el elemento raíz -->
  <xs:element name="persona">
    <xs:complexType>
      <xs:sequence>
        <!-- Elemento 'nombre' debe ser texto -->
        <xs:element name="nombre" type="xs:string" />
        <!-- Elemento 'edad' debe ser un número entero -->
        <xs:element name="edad" type="xs:integer" />
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

Explicación Paso a Paso:

1. El elemento raíz:

En el XSD, el elemento `<xs:element name="persona">` define que el XML debe comenzar con un elemento llamado `<persona>`.

2. Elementos hijos:

- `<nombre>` debe ser texto (`type="xs:string"`).
- `<edad>` debe ser un número entero (`type="xs:integer"`).

3. Orden y estructura:

Los elementos `<nombre>` y `<edad>` deben aparecer en ese orden exacto.

Cómo funciona la validación:

- Si escribes un XML que no sigue estas reglas (por ejemplo, si olvidas `<edad>` o pones texto en vez de un número), el validador te dirá que el documento no es válido.

¿Por qué usar XML Schema?

- Asegura que los datos en el XML estén bien estructurados y correctos.
- Es más detallado y flexible que los DTD.
- Permite definir tipos de datos (texto, números, fechas, etc.).

En resumen, **los lenguajes de marcado como XML proporcionan una forma estructurada y estándar de representar datos**, lo que los hace fundamentales para la interoperabilidad entre aplicaciones y sistemas.